

SWP391 CAPSTONE · 2026

Manga Creation Workflow & Publishing Management System

An internal production & publishing platform for a manga studio
— *it is an operational tool, not a manga reader.*

5 roles · 29 tables · 66 REST endpoints · in-browser AI Studio

Agenda

01

The Product & Why

What it solves and the operating model

02

Architecture & Technology

Monorepo, stack, request flow, workflow engine

03

Data Model

29 tables, relationships, state machines

04

Security & Access Control

JWT/OAuth and role-based permissions

05

The Five Roles in Action

What each role does, screen by screen

06

Key Feature Flows

Proposal, production, governance, disputes

07

Studio & On-Device AI

In-browser drawing + ONNX assists

08

Quality & Roadmap

Verification, backlog, live demo

01

The Product & Why

Why a studio needs this, and how the pipeline works end to end

The problem we solve

- A manga studio coordinates many people around one series
- Mangaka pitch series; a board decides what gets greenlit
- Assistants do paid production work (backgrounds, characters, effects...)
- Editors review chapters; the board governs which series continue
- Today this lives in spreadsheets, chat and email — no single source of truth

Lost work & versions

Files and approvals scattered across tools

No status truth

Nobody knows a chapter's real state

Payment disputes

Earnings computed by hand, hard to audit

What it is — and what it is not

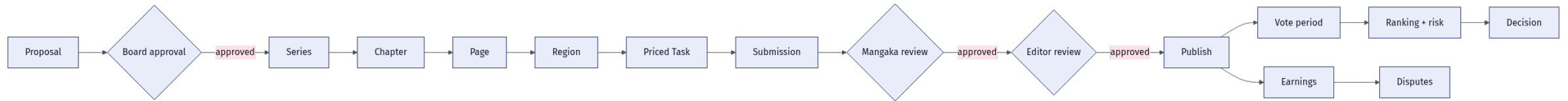
IT IS

- An internal operations tool for studio staff
- A workflow engine for the production pipeline
- A role-based system (5 roles, RBAC)
- A pricing + earnings + dispute ledger
- A browser drawing Studio with optional AI

IT IS NOT

- A public manga reader / consumer app
- A payment gateway (it records, doesn't charge)
- A cloud AI service — AI runs on-device
- A CMS for marketing the manga
- Dependent on any paid third-party API

The end-to-end pipeline



- One continuous flow from idea to governance to payment
- Each stage has an explicit status; illegal transitions are rejected server-side
- Approval auto-creates a Series; publishing schedules a release; voting drives ranking & risk; decisions change a series' fate

The five roles



Mangaka

Author & produce series

Proposals · Series · Tasks · Review



Assistant

Execute paid production tasks

My Tasks · Studio · Earnings



Tantou Editor

Review & annotate chapters

Review queue · Annotations



Editorial Board

Govern: approve, vote, decide

Proposals · Assign · Rankings



Admin

Operate users & disputes

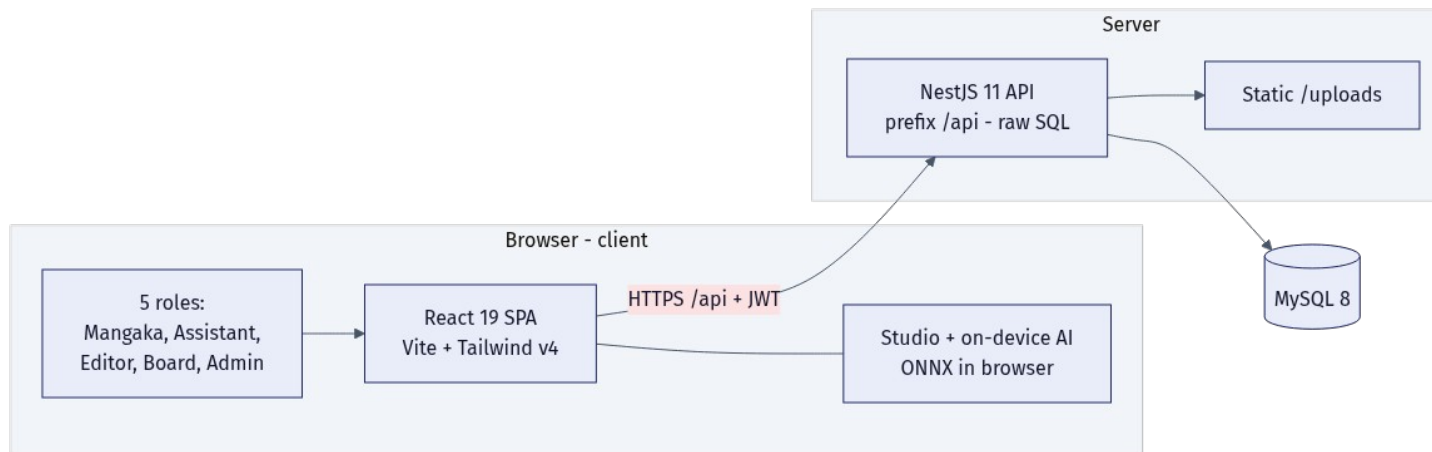
Console · Disputes

02

Architecture & Technology

How the monorepo, API, database and browser pieces fit together

System architecture



- Single-page React app talks to one NestJS API
- API exposes everything under /api; serves uploads statically
- Raw SQL via mysql2 — no ORM, full control of queries
- Shared TypeScript enums = one source of truth for both sides
- The Studio + AI run entirely in the browser

Technology stack

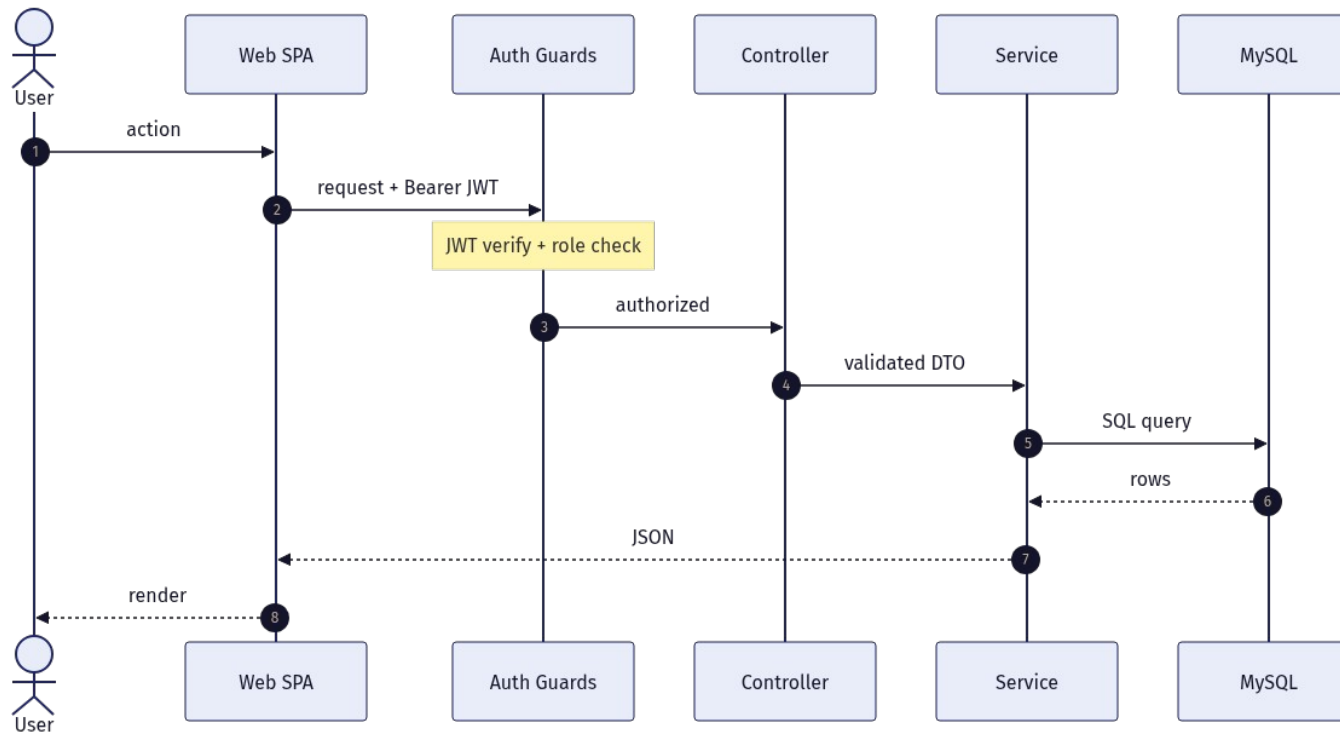
Layer	Technology	Version
Web app	React + Vite + TypeScript + Tailwind	19.2 · 8 · ~6.0 · v4.3
Routing / HTTP	react-router-dom · axios	7.16 · 1.16
API	NestJS + TypeScript	11 · 5.7
Database access	mysql2 (raw SQL, no ORM)	3.22
Auth	@nestjs/jwt · passport (jwt + google)	11 · —
Shared	TS enums + state-machine transition maps	workspace pkg
Drawing core	AssemblyScript → WebAssembly	canvas-wasm
On-device AI	onnxruntime-web (YOLO · SAM · DeOldify)	1.26
Tests	jest (API) · vitest + RTL (web)	30 · 4

Monorepo & shared types

```
manga-workflow/ (pnpm workspace)
├─ apps/
│  └─ web/      React 19 SPA (Vite, Tailwind v4)
│     └─ api/    NestJS 11   (raw SQL, /api)
├─ packages/
│  └─ shared/    enums + *_TRANSITIONS + canTransition()
│     └─ canvas-wasm/ AssemblyScript → WASM
└─ db/          MySQL 8 schema + seed (Docker)
```

- One repo, four packages, clear boundaries
- Build order: shared + wasm → then apps
- Shared enums prevent web/API status drift
- DB schema + seed live in db/ and run via Docker

Request lifecycle



- Every authenticated call follows the same path
- JWT verified, then @Roles checked by a guard
- DTOs validated by a global ValidationPipe
- Service runs parameterized SQL via DbService
- Consistent JSON + Nest error shape back

The workflow engine: state machines

- Status enums + transition maps are the rules engine
- Six entities have explicit, enforced transition maps
- Services call `canTransition()` before any status write
- Illegal transition → HTTP 400, never a bad state

```
export function canTransition<T extends string>(
  map: Record<T, T[]>, from: T, to: T,
): boolean {
  return map[from]?.includes(to) ?? false;
}
```

Proposal

transition map

Chapter

transition map

Page

transition map

Task

transition map

Submission

transition map

Earning Dispute

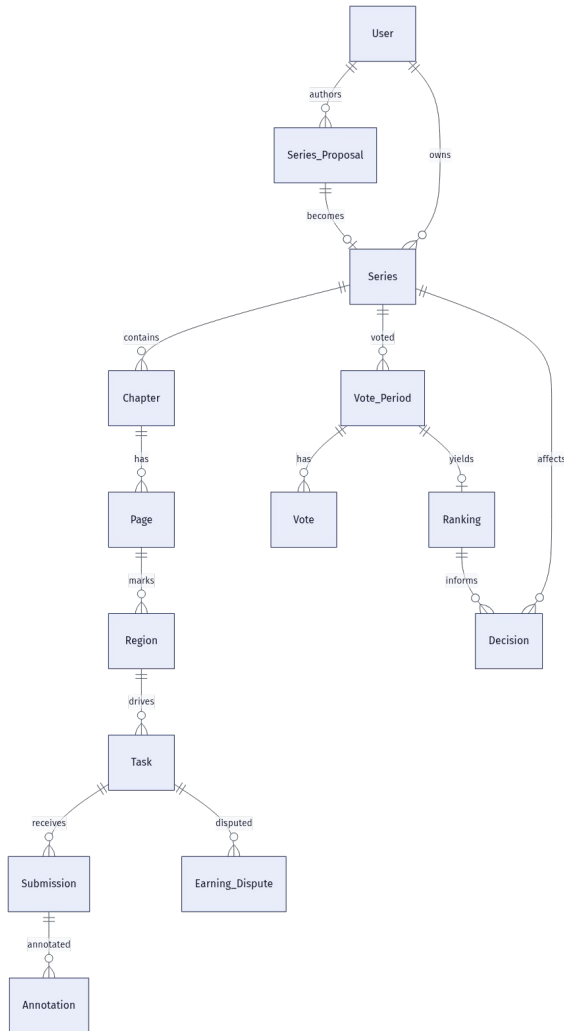
transition map

03

Data Model

29 tables across nine domains, with enforced lifecycles

Domain map



- Nine domains, 29 tables, foreign-key integrity
- Identity & profiles: User + 4 role profiles
- Series lifecycle: Proposal → Series (+ editor assignment)
- Production: Chapter → Page → Region → Task → Submission
- Editorial feedback: polymorphic Annotation
- Governance: Vote Period → Vote → Ranking → Decision
- Money: Assistant earnings + Earning Dispute
- Cross-cutting: Notification, Audit Log, System Config

Core entities

User + Profiles

One identity, four role-specific profile tables
(earnings live on Assistant_Profile)

Series_Proposal → Series

Approval converts a pitch into a live series with a frequency

Chapter · Page · Page_Version

Versioned page images grouped into chapters

Region · Task · Task_Price_Rule

Regions are priced into tasks by region type

Submission · Annotation

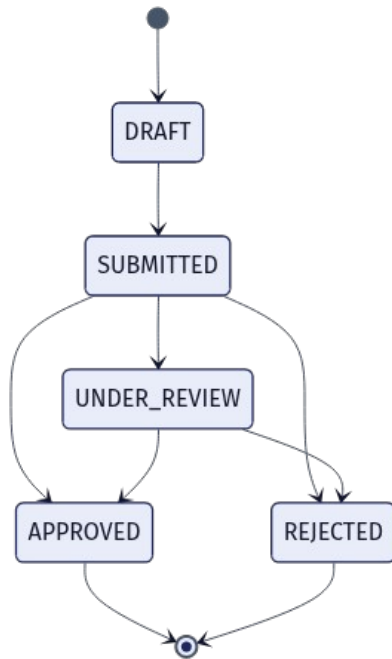
Assistant work + polymorphic editorial notes

Vote_Period · Vote · Ranking · Decision

The governance loop, snapshotted per period

Enforced state machines

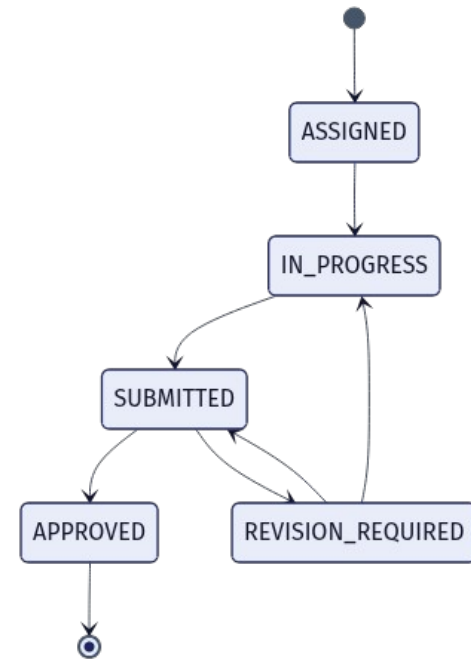
Proposal



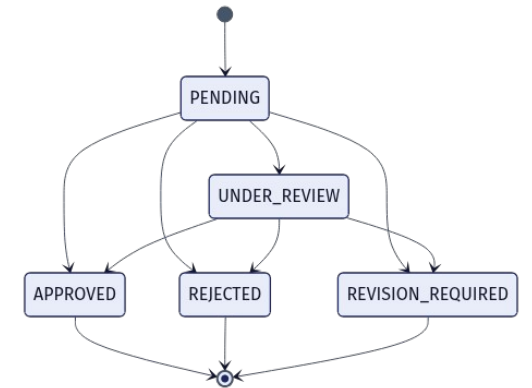
Chapter



Task



Submission

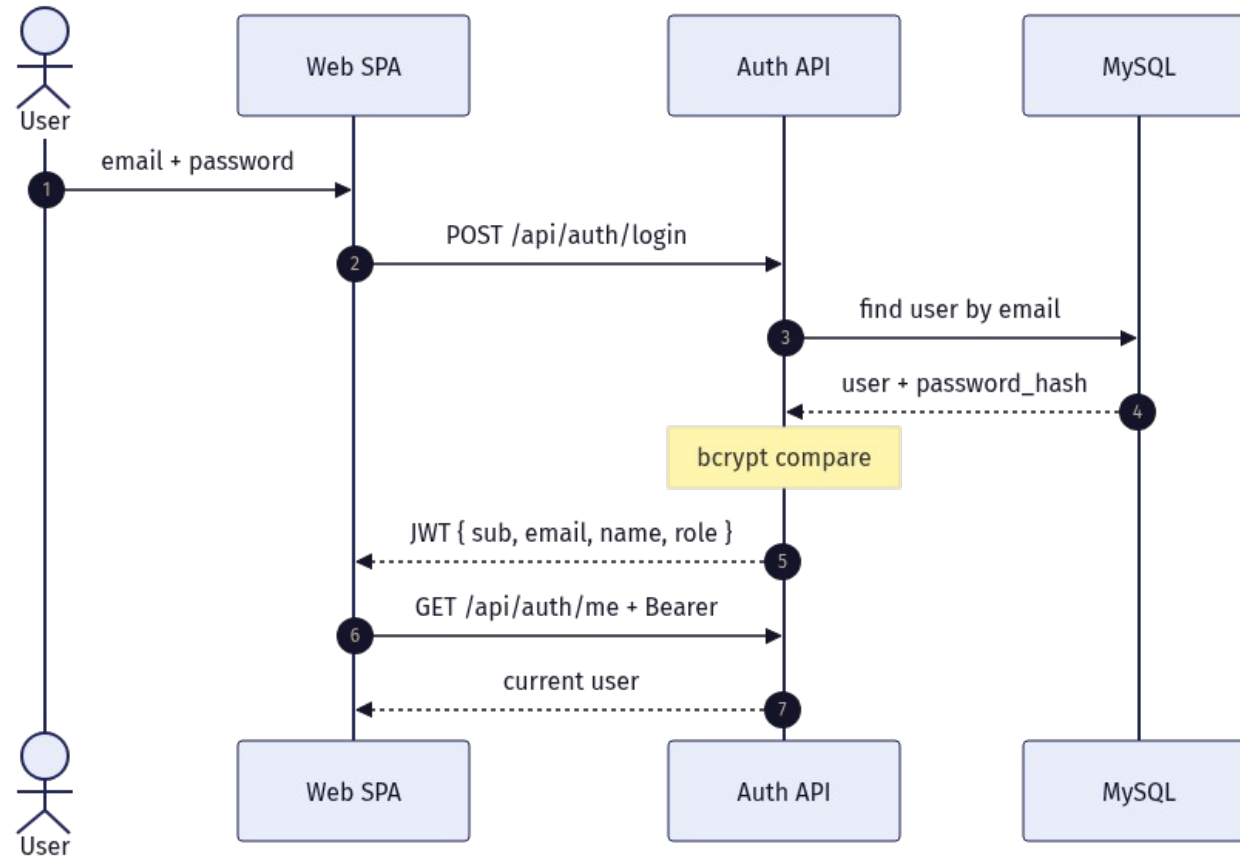


04

Security & Access Control

Authentication, RBAC, and data-level ownership

Authentication



- Two ways in: local password + Google OAuth
- Passwords hashed with bcrypt (nullable for OAuth-only)
- JWT carries { sub, email, name, role }
- Web sends it as an Authorization: Bearer header
- Account must be activated to sign in

Authorization — permission matrix

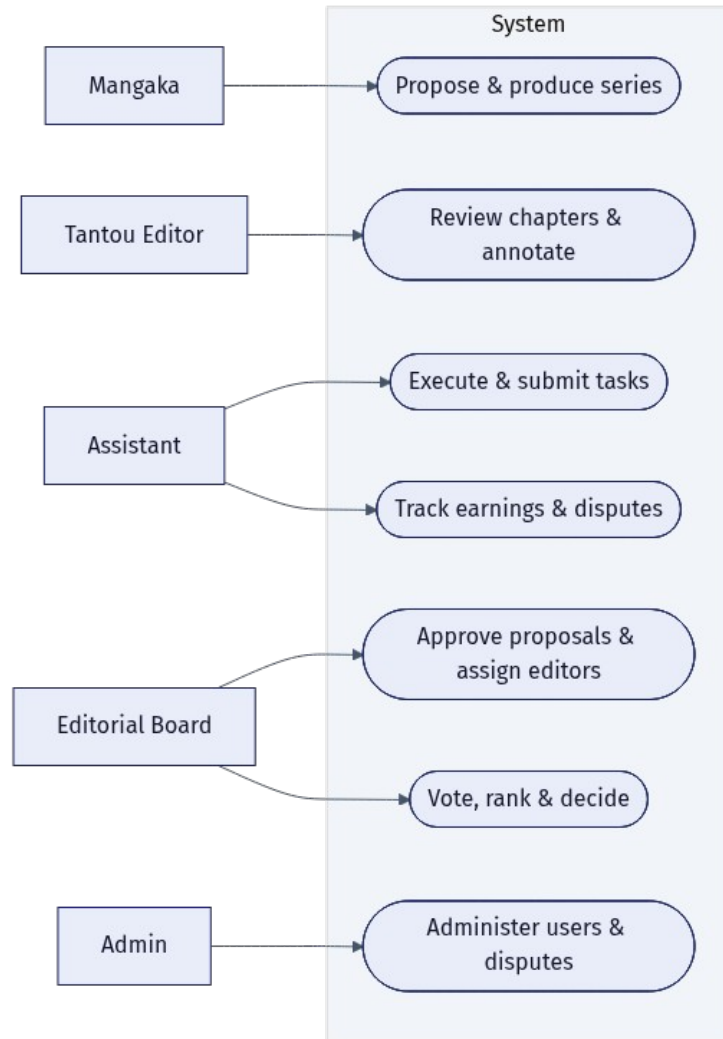
Capability / endpoint group	Mangaka	Assist.	Editor	Board	Admin	Public
Login · Google OAuth						✓
Proposals — create / mine	✓					
Proposals — review / decide				✓		
Series — assign / unassign editor				✓		
Chapters · Pages · Regions · Tasks	✓					
Tasks — mine / start		✓				
Submissions — submit		✓				
Submissions — review	✓					
Chapter review · Annotations			✓			
Vote periods · Votes · Decisions				✓		
Earnings — mine / disputes (open)		✓				
Disputes — list / review / resolve					✓	
Admin — users / overview					✓	

05

The Five Roles in Action

What each person sees and does, screen by screen

Use-case overview



- Each actor has a focused set of use cases
- **Mangaka**: propose & produce a series
- **Assistant**: execute & submit tasks; track earnings
- **Tantou Editor**: review chapters & annotate
- **Board**: approve, assign, vote, rank, decide
- **Admin**: administer users & disputes
- **The System** also acts: notify, price, accrue, rank

Mangaka

漫画家 • *Series Author*

WHAT THEY DO

- Pitch and submit series proposals
- Create chapters, pages, and regions
- Create priced tasks and assign assistants
- Review submissions: approve or request revision

NAVIGATION

- Tổng quan /
- Đề xuất /proposals
- Series /series
- Chờ duyệt /review

KEY ENDPOINTS

POST /api/proposals

POST /api/chapters • /pages • /regions

POST /api/tasks (auto-priced)

PATCH /api/submissions/:id/review

DRIVES STATUSES

Proposal • Chapter • Task • Submission review

Assistant

アシスタント • *Production Artist*

WHAT THEY DO

- Pick up and start assigned tasks
- Draw in the in-browser Studio (optional AI)
- Upload and submit work for review
- Track earnings; file a dispute if needed

NAVIGATION

- Tổng quan /
- Việc của tôi /my-tasks
- Thu nhập /earnings
- Studio /studio/region/:taskId

KEY ENDPOINTS

GET /api/tasks/mine • PATCH /:id/start
POST /api/uploads • /api/submissions
GET /api/earnings/mine
POST /api/disputes

DRIVES STATUSES

Task start/submit • Submission • Dispute (open)

Tantou Editor

担当編集 • *Series Editor*

WHAT THEY DO

- See a queue scoped to assigned series only
- Page through a chapter under review
- Leave categorized annotations on pages
- Approve the chapter or request changes

NAVIGATION

- Tổng quan /
- Duyệt chương /editor/review
- Chapter review /editor/review/:id

KEY ENDPOINTS

GET /api/chapters/review-queue

GET /api/chapters/:id/pages

POST /api/annotations • PATCH /:id/resolve

PATCH /api/chapters/:id/editor-review

DRIVES STATUSES

Chapter → EDITOR_APPROVED / IN_PROGRESS

Editorial Board

編集委員会 · Governance

WHAT THEY DO

- Approve or reject proposals (→ Series)
- Assign / unassign the Tantou editor
- Open vote periods and cast scored votes
- Close periods → ranking; record decisions

NAVIGATION

- Tổng quan /
- Duyệt đề xuất /board/proposals
- Phân công BT /board/series
- Xếp hạng /board/rankings

KEY ENDPOINTS

PATCH /api/proposals/:id/decision
PUT/DELETE /api/series/:id/editor
POST /api/vote-periods · /votes · /:id/close
POST /api/decisions

DRIVES STATUSES

Proposal · Vote Period · Series status

Admin

システム管理者 • Platform Admin

WHAT THEY DO

- See a system overview / metrics
- Activate users and assign roles
- Guarded so the last admin can't be removed
- Adjudicate earning disputes (adjust amount)

NAVIGATION

- Tổng quan /
- Quản trị /admin
- Khiếu nại /admin/disputes

KEY ENDPOINTS

GET /api/admin/overview • /users

PATCH /api/admin/users/:id

GET /api/disputes

PATCH /api/disputes/:id/review • /resolve

DRIVES STATUSES

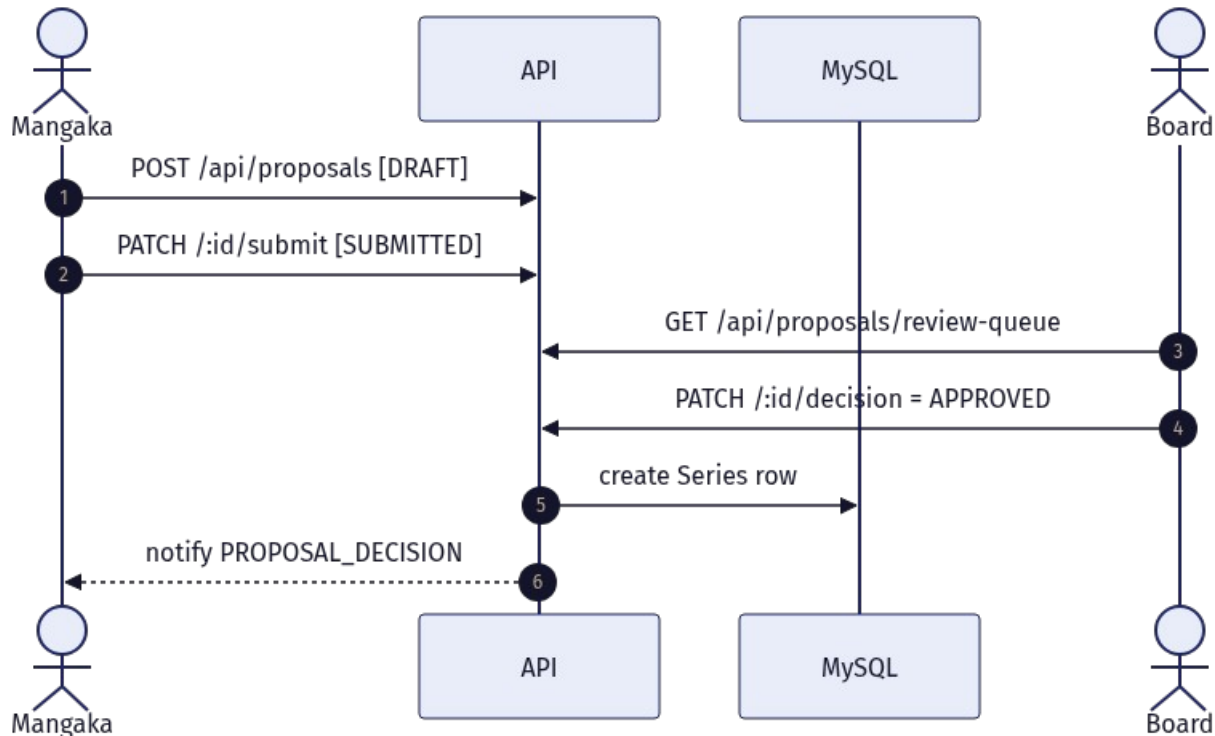
User activation/role • Earning Dispute

06

Key Feature Flows

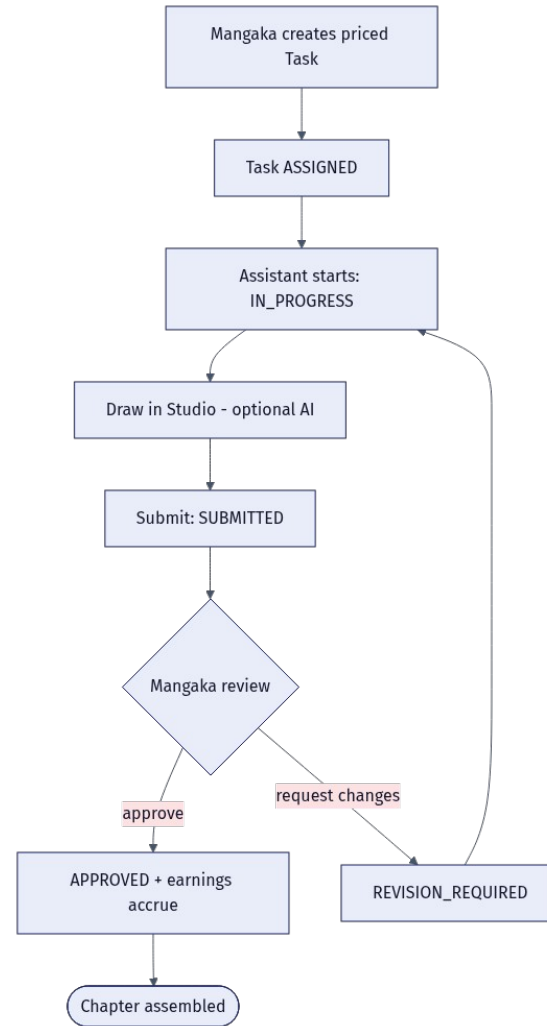
Five flows that exercise the whole system

Proposal → approval → Series



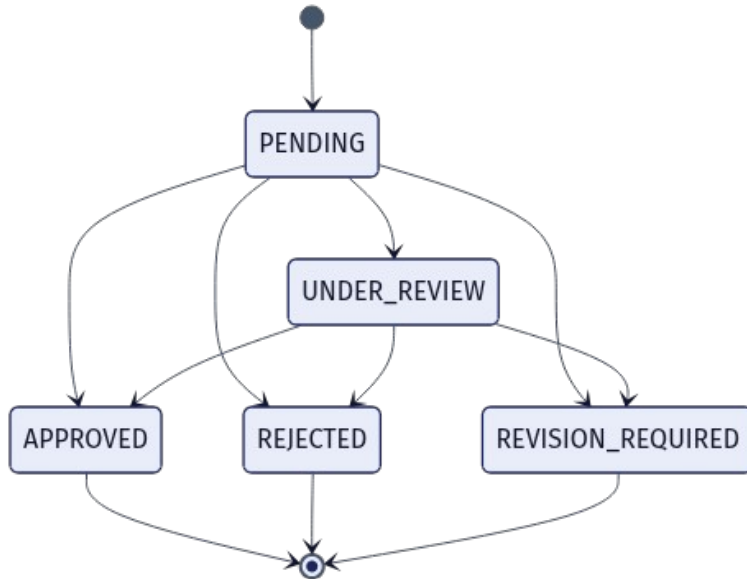
- Board approval creates the Series
- Mangaka drafts then submits a proposal
- Board reviews the queue and decides
- **APPROVED** auto-inserts a Series row
- Mangaka is notified of the decision

Production loop



- Tasks iterate until approved
- Task auto-priced from Task_Price_Rule
- Assistant starts, draws, submits
- Mangaka approves → earnings accrue
- Or requests changes → loops back

Submission review & earnings accrual



- Approval is the only path to payment
- Mangaka reviews from a queue
- APPROVED → total_earnings += task payment
- REVISION_REQUIRED → assistant iterates
- Every outcome notifies the assistant

Auto

task pricing by region type

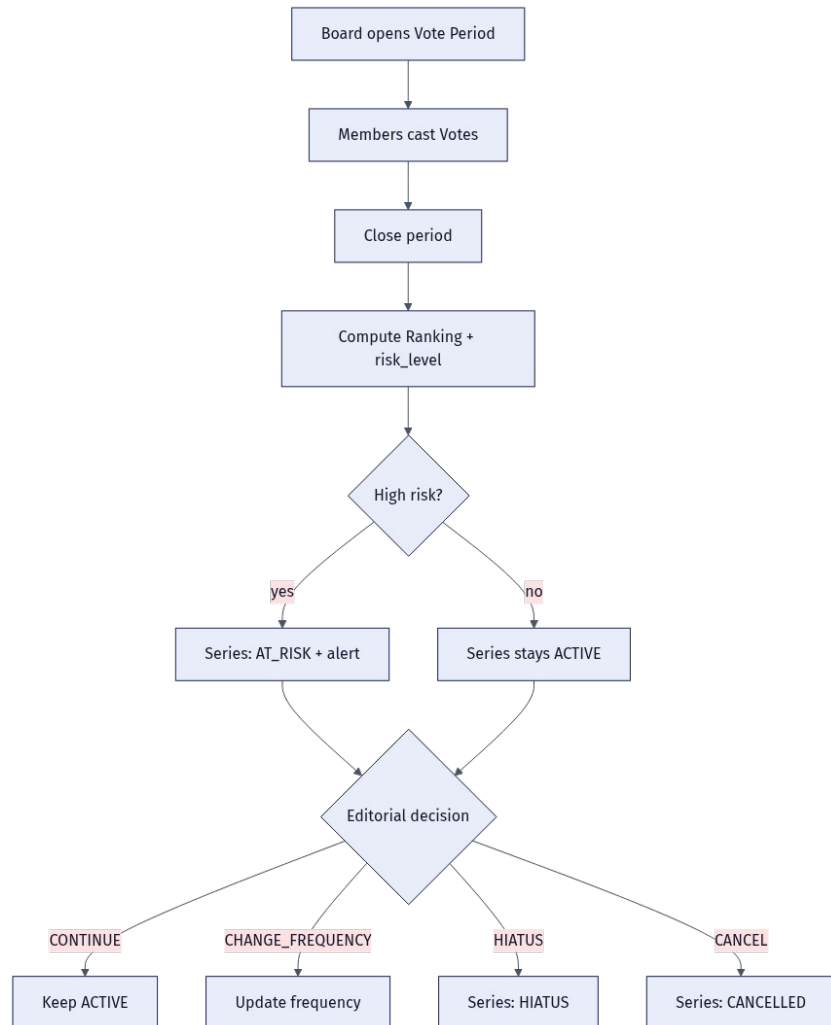
+\$

accrues only on APPROVED



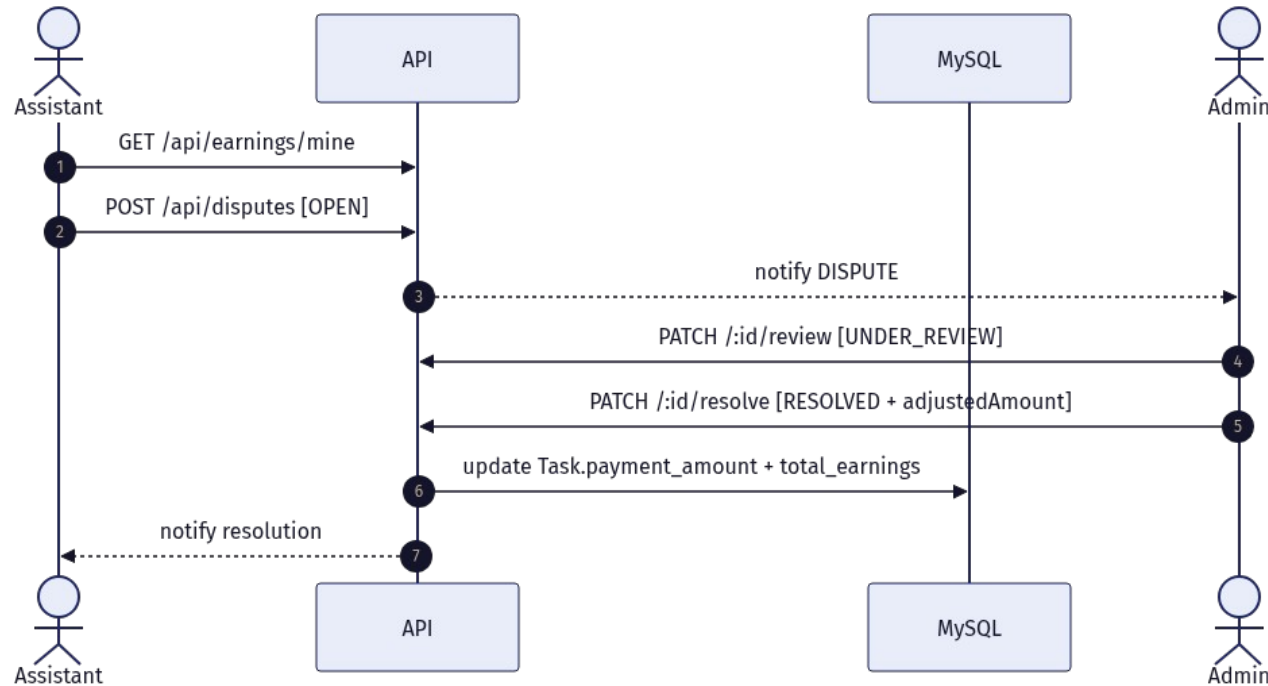
disputes adjust by delta

Governance: vote → ranking → decision



- Voting steers a series' future
- One vote per board member per period
- Closing computes ranking + risk level
- High risk flips Series to AT_RISK
- Decision: continue / change / hiatus / cancel

Earnings & disputes



- A fair, auditable money trail
- Assistant views earnings by task
- Opens a dispute on an approved task
- Admins are notified; one reviews
- Resolve can adjust pay + re-accrue

07

Studio & On-Device AI

An in-browser drawing tool with private, zero-cost AI

The in-browser Studio

Layers & document

Non-destructive layer stack with history

Brush engine

Pressure-style brush, WASM-accelerated pixels

Selection & transform

Rect, ellipse, lasso, wand, move/scale

Manga tools

Panels, tone, lines, symmetry

Text & bubbles

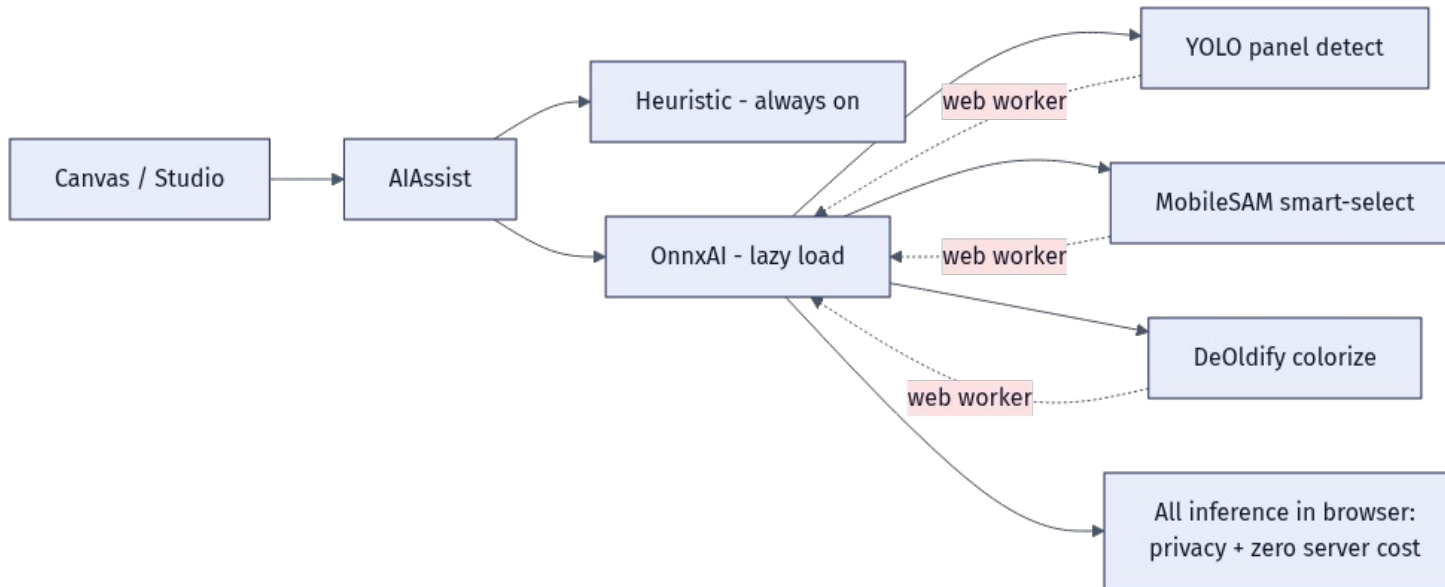
Speech bubbles and text layers

Import / export

Load references, export the artwork

Pixel operations are compiled from AssemblyScript to WebAssembly for native-like speed — all in the browser.

On-device AI assists



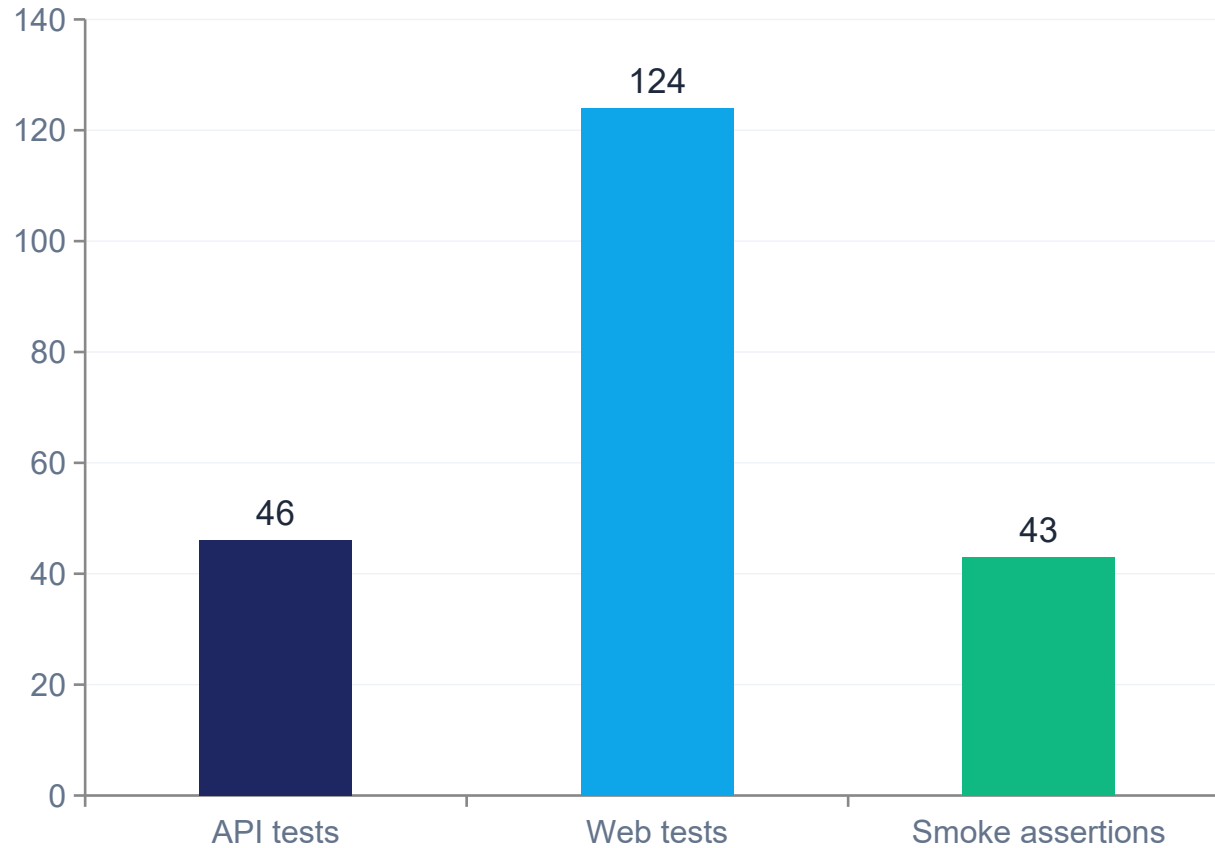
- **ONNX Runtime Web** — inference in the browser
- **Panel detection (YOLO)**
- **Smart selection (MobileSAM)**
- **Colorization (DeOldify)**
- **Heuristic fallback if models unavailable**
- **Privacy: images never leave the client**
- **Cost: zero per-inference server cost**

08

Quality & Roadmap

How we verified it, what's next, and a live demo

Verification & quality



- Green across the board
- API: 46 jest tests + build pass
- Web: 124 vitest tests + tsc -b + build pass
- 3 live end-to-end smokes against real DB
- 0 failures; integration via PR #5

Roadmap & backlog

SHIPPED

- Full pipeline: proposal → decision → earnings
- All 5 roles, RBAC, notifications
- In-browser Studio + on-device AI
- 29-table schema, enforced state machines

NEXT

- Wire Audit_Log + System_Config tables
- Profile-edit pages; Task_Price_Rule admin UI
- Production hardening: object storage for uploads
- Refresh-token rotation + rate limiting

Live demo script

1. Board logs in → approves a proposal → a Series appears
1. Board assigns a Tantou editor to the new series
1. Mangaka builds a chapter, marks regions, creates priced tasks
1. Assistant starts a task, draws in Studio, submits work
1. Mangaka approves → assistant's earnings increase
1. Editor approves the chapter → publish → board votes → decides

DEMO LOGINS

mangaka

dungminer69@gmail.com

assistant

mai.assistant@inkframe.studio

editor

hiroshi.editor@inkframe.studio

board

yamamoto.board@inkframe.studio

admin

admin@inkframe.studio

password: *Dung123456@ (local demo seed)*

Thank you

Questions & live demo

Full technical docs: [documents/](#) — overview, architecture, database design, API reference (66 endpoints), diagrams, and per-role guides.